# Dynamic Programming

# Rod Cutting

# Readings and problems to solve

- CLR: Chapter 15, section 15.1
- Similar/Extra problems:
- Question 1:
  - https://leetcode.com/problems/minimum-cost-to-cut-a-stick/description/
- Question 2:
  - https://leetcode.com/problems/ways-to-split-array-into-three-subarrays/
- Question 3
- Binomial coefficient Design an efficient algorithm for computing the binomial coefficient C(n, k) that uses no multiplications. What are the time and space efficiencies of your algorithm?

# Rod cutting

- **Input:**
  - a rod of length n inches
  - A table of prices p
- **Output**:
  - You may cut the rod into pieces; revenue is the sum of prices of pieces.
  - Goal: maximize revenue

| LENGTH | PRICE |
|---|---|
| 1 | 1 |
| 2 | 5 |
| 3 | 8 |
| 4 | 9 |
| 5 | 10 |
| 6 | 17 |
| 7 | 17 |
| 8 | 20 |
| 9 | 24 |
| 10 | 30 |

# Rod cutting: Example

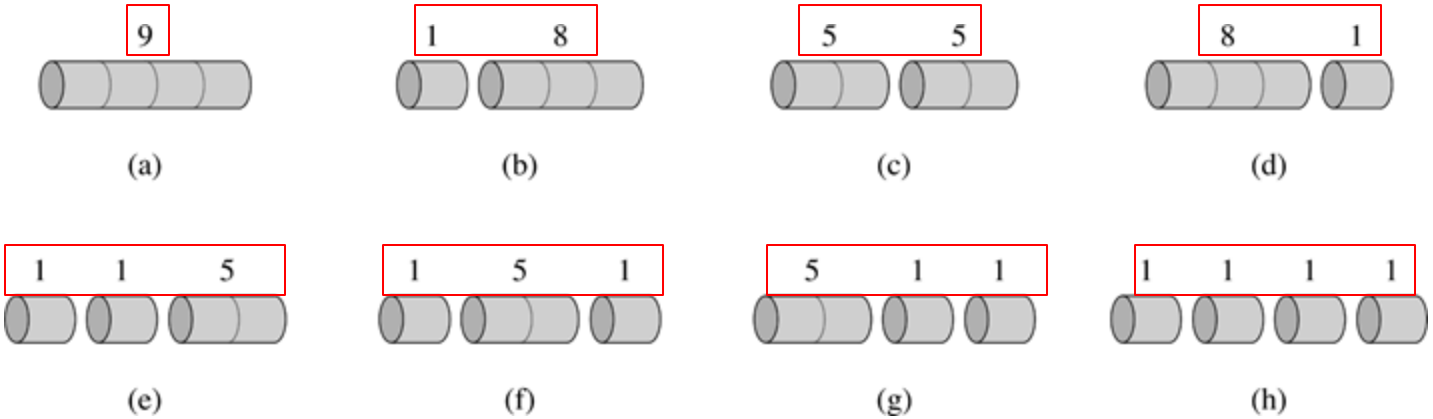
- How many possible ways of cutting up a rod of **length 4**

| LENGTH | PRICE |
|---|---|
| 1 | 1 |
| 2 | 5 |
| 3 | 8 |
| 4 | 9 |

# Rod cutting: Example

| LENGTH | PRICE |
|---|---|
| 1 | 1 |
| 2 | 5 |
| 3 | 8 |
| 4 | 9 |

- The are 8 possible ways of cutting up a rod of **length 4**
  - Above each piece is the value of that piece
  - The revenue for each strategy is sum of the values of pieces
- Optimal strategy is (c)



# Rod cutting: Brute Force

- The rod has length n
- At distance i from left, there are two options:
  - Cut at distance i
  - No cut at distance i
- Runtime: There are $2^{n-1}$ possibilities, we check all and take the maximum
  - Runtime: $O(2^n)$

# Rod Cutting

- Can we solve it using divide-and-conquer / recursive approach?

| LENGTH | PRICE |
|---|---|
| 1 | 1 |
| 2 | 5 |
| 3 | 8 |
| 4 | 9 |
| 5 | 10 |
| 6 | 17 |
| 7 | 17 |
| 8 | 20 |
| 9 | 24 |
| 10 | 30 |

# Rod Cutting: Finding subproblems and Recursive formula

$r_n \rightarrow$ max revenue for length $n$

$r_j \rightarrow$ for length $j < n$




$r_n = p[i] + r_{n-i}$ subproblem

# Rod Cutting: Finding subproblems and Recursive formula

- $\mathbf{r_n}$ : The maximum revenue for length n
- **Subproblem:** $\mathbf{r_j}$ : The maximum revenue for smaller rod length j, where j < n

- $\mathbf{r_n = max ( \ p[i] + r_{n-i} \ )}$ **where 1 ≤ i ≤ n**
- Base case: $r_0$ is when the rod is not cut $\mathbf{r_0} = p_n$

# Rod Cutting: Finding Recursive Formula

- $\mathbf{r_n}$ : The maximum revenue for length n
- Base case: $r_0$ is when the rod is not cut $\mathbf{r_0}= p_n$
- Recursive case:
  - Make the first cut at some length i
  - Solve the subproblem recursively, subproblem is $\mathbf{r_{n-i}}$
  - Pick the length i so that p[i] + $r_{n-i}$ is maximum
    - What should be the value of i? We don't know, So we try all possible cases

$$r_n = max_{1 \leq i \leq n} (p_i + r_{n-i})$$

# Rod cutting: Why DP can be used?

- Rod cutting problem has optimal substructure property
  - We can break down the problem into smaller subproblems, and the solution to the original problem can be built from the solutions to these subproblems.
    - The optimal solution for length n ($\mathbf{r_n}$) contains the optimal solution to $\mathbf{r_{n-i}}$
    - If $\mathbf{r_{n-i}}$ is not optimal, we can replace it with the optimal value, and get a better value for $\mathbf{r_n}$
    - **Each piece of rod is independent of the other piece after a cut: subproblems are independent**
- Overlapping subproblems:
  - The same subproblems appear multiple times, making it efficient to store and reuse results.
- Number of different subproblems are polynomials

Therefore, dynamic programming is an effective way to solve this problem

# Rod cutting: Recursive Implementation

**Runtime:**

T(n) = T(n-1) + T(n-2) + T(n-3) + T(n-4) + … + T(1) + T(0)

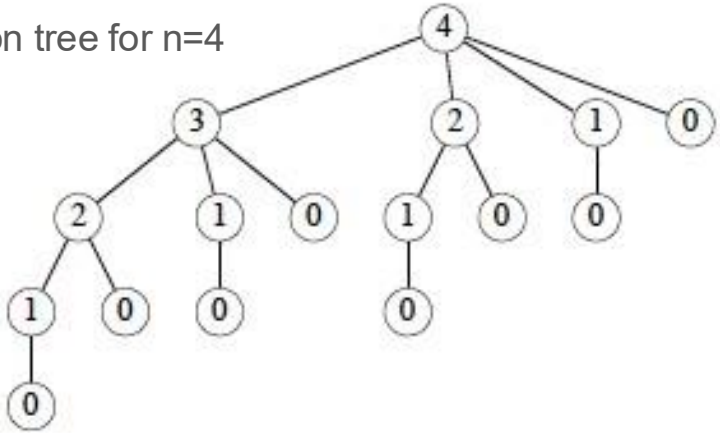
T(0) = 1

It is exponential

$$T(n) = 1 + \sum_{i=0}^{n-1} T(i)$$

```
def cut_rod(n)
    if n==0:
        return 0
    q = negative infinity
    for i=1 to n:
        q = max (q, p[i]+ cut_rod(n-i))
    return q
```

The recursion tree for n=4



# Rod cutting: Recursive Implementation: Runtime Analysis

- Calculating T(n): Recurrence relation with a full history
- Using substitution method

$$T(n) = 1 + \sum_{i=0}^{n-1} T(i)$$

$$T(n+1) = 1 + \sum_{i=0}^{n} T(i)$$

$$T(n+1) - T(n) = T(n)$$

$$T(n+1) = 2T(n)$$

$$T(n+1) = 2 * 2T(n-1) = 2 * 2 * 2T(n-2) = 2 * 2 * 2 * 2T(n-3) = 2^n T(1) = 2^n$$

$$Guess : T(n) = 2^n$$

# Rod cutting: Implementation

- Recursive implementation:
  - Runtime: Exponential
- Recursive implementation with memoization: Dynamic Programming
  - Top-down dynamic programming
  - Runtime: Polynomial
- Iterative implementation: Dynamic Programming
  - Runtime: Polynomial

# Rod cutting: DP: top-down with memoization

- Just modify the previous algorithm to save the result of each subproblem
- **Memoized**: It remembers the results computed previously
- You can define r as a global variable or as a local variable and pass it as an argument to each call of the function (check CLR for that)

```
r = [None, None, None, None, … ,None] #r has n+1 elements
def cr_memoized(n)
    if r[n] is not None:
        return r[n]
    If n==0:
        q = 0
    else
         q = negative infinity
         for i=1 to n:
             q = max (q, p[i]+ cr_memoized(n-i))
         r[n] = q
     return q
```

# Rod cutting: DP: top-down with memoization

- **Runtime: $\Theta(n^2)$**
  - **# subproblems x (time for each subproblem)**
  - **How many subproblems: n**
    - Each subproblem is solved only once
  - How much time for each subproblem?
    - To solve a problem of size i the for loop iterates i times
  - Therefore, enough to count the number of iteration of the for loop for each problem:
  - $1+2+...+n = n(n+1)/2 = \Theta(n^2)$

# Rod cutting: DP: bottom-up

- subproblem j needs all subproblems smaller than j
- fill the table by filling the smaller indices first
- Runtime: $\Theta(n^2)$

**Cut-rod-bottom-up (p, 4)**

| | $r_0$ | $r_1$ | $r_2$ | $r_3$ | $r_4$ |
|---|---|---|---|---|---|
| r[i] | 0 | 1 | 5 | 8 | 10 |

| LENGTH | PRICE |
|---|---|
| 1 | 1 |
| 2 | 5 |
| 3 | 8 |
| 4 | 9 |

```
def cut-rod-bottom-up(n)
    r = [None, None, None, None, … ,None]
    #r has n+1 elements
    r[0] = 0
    for j=1 to n
        q = negative infinity
        for i=1 to j: #i is the position of the first cut
            q = max (q, p[i]+ r[j-i])
        r[j] = q
    return r[n]
```

# Rod cutting: Reconstructing a Solution

- Return the optimal solution in addition to the value of the optimal solution
- Arrays s says where to cut the array (at what length)
- To find the complete solution, after printing s[n] we need to look at the position n-s[n] in array s

# Rod cutting: Reconstructing a Solution

| LENGTH | PRICE |
|---|---|
| 1 | 1 |
| 2 | 5 |
| 3 | 8 |
| 4 | 9 |

```
def extended-cut-rod-bottom-up(p, n)
      r = [None, None, None, None, … ,None]
      s = [None, None, None, None, … ,None]
      #r has n+1 elements
      r[0] = 0
      for j=1 to n
          q = negative infinity
          for i=1 to j:
              #q = max (q, p[i]+ r[j-i])
              If q < p[i] + r[j-i]
                  q = p[i] + r[j-i]
                  s[j] = i
          r[j] = q
      return r, s

print_solution(p, n)
      r,s = extended-cut-rod-bottom-up(p, n)
      while n > 0
          print s[n]
          n = n - s[n]
```

**Cut-rod-bottom-up (p, 4)**

| | $r_0$ | $r_1$ | $r_2$ | $r_3$ | $r_4$ |
|---|---|---|---|---|---|
| r[i] | 0 | 1 | 5 | 8 | 10 |
| s[i] | | 1 | 2 | 3 | 2 |

# Rod cutting: Dynamic Programming: Summary

- Runtime: $\Theta(n^2)$
- Space: $\Theta(n)$

# Recipe for writing a dynamic programming

1. Identify subproblems
    a. Check to see if the problem has the optimal substructure property and identify the optimal substructure
2. Find a recursive formulation for solving the larger subproblems, given solution to the smaller subproblems
3. Find the final solution
4. Implement the recursive formulation using memoized top-down or bottom-up approach
    a. Which one is preferred?
        i. Top-down version is harder to analyze its runtime
        ii. Overhead of recursive approach takes more time
5. If needed, modify and extend the algorithm so that it returns the actual solution as well as the value of the solution